Public

FACULDADE DE INFORMÁTICA E ADMINISTRAÇÃO PAULISTA

Gabriel Lacerda Covello Arimatéa 556391

Geovana Carvalho Pederneschi 559092

Gustavo Andrade de Sousa 559069

Lucas Santos Rodrigues 556891

Mayene Gabrielle Aragão Padilha Doria 558858

Thais Helena Ferreira Vieira 552387

RELATÓRIO
Fase 1 – Documentação detecção de EPI

ENGENHARIA DA COMPUTAÇÃO - 3ECR
SPI/Metaindústria

Public

SUMÁRIO

1. Introdução....4
1.1 Contextualização da fase....4
1.2 Requisitos mínimos....4
2. Machine Learning....5
2.1 Desenvolvimento....5
2.1.1 Revisão teórica....5
2.1.2 Dados....5
2.1.3 Metodologia....6
2.1.4 Ferramentas....6
2.1.5 Pipeline de implementação....6
2.2 Resultados....6
2.2.1 Métricas de avaliação....6
2.2.2 Desempenho do(s) modelo(s)....7
2.2.3 Análise de erros....7
2.3 Evidências....7
2.3.1 Visualizações de suporte....7
2.3.2 Exemplos práticos....8
2.3.3 Logs e reprodutibilidade....8
3. Frontend....8
3.1 Desenvolvimento....8
3.1.1 Revisão teórica....8
3.1.2 Requisitos e escopo da interface....8
3.1.3 Arquitetura e componentes....8
3.1.4 Integração com backend....9
3.1.5 Ferramentas....9
3.1.6 Pipeline de implementação....9
3.2 Resultados....9
3.2.1 Critérios de avaliação....9
3.2.2 Funcionalidades entregues....9
3.2.3 Análise de problemas....9
3.3 Evidências....10
3.3.1 Prints e protótipos de telas....10
3.3.2 Exemplos práticos....10
3.3.3 Logs e reprodutibilidade....10

Public

4. Backend..... 10
4.1 Desenvolvimento..... 10
4.1.1 Revisão teórica..... 10
4.1.2 Requisitos e escopo dos serviços..... 10
4.1.3 Arquitetura..... 10
4.1.4 Integração com o modelo de ML..... 11
4.1.5 Ferramentas..... 11
4.1.6 Pipeline de implementação..... 11
4.2 Resultados..... 11
4.2.1 Métricas de avaliação..... 11
4.2.2 Desempenho dos serviços..... 11
4.2.3 Análise de erros..... 11
4.3 Evidências..... 11
4.3.1 Visualizações de suporte..... 11
4.3.2 Exemplos práticos..... 12
4.3.3 Logs e reprodutibilidade..... 12
5. Conclusão..... 13
5.1 Próximos passos..... 13
5.2 Considerações finais..... 13

Public

1. Introdução

1.1 Contextualização da fase

O projeto Codexis nasce da constatação de que a fiscalização manual de segurança do trabalho não acompanha a realidade das linhas de produção industriais. Em 2023, o Brasil registrou mais de 580 mil acidentes de trabalho, dos quais aproximadamente 30% envolveram falha no uso correto de Equipamentos de Proteção Individual (EPIs), gerando um custo direto e indireto estimado em R$ 70 bilhões por ano com Segurança e Saúde no Trabalho (SST), segundo dados do Ministério do Trabalho e Emprego (MTE/Smartlab) e da Organização Internacional do Trabalho (OIT).

Essa ineficiência decorre de quatro fatores estruturais: inspeções presenciais pontuais que não cobrem turnos completos; câmeras de CFTV que apenas gravam, sem qualquer análise automatizada; registros e auditorias dependentes de planilhas e papel; e identificação tardia de não conformidades, quando o acidente já ocorreu.

Diante desse cenário, a fase atual (1ª Mentoria, realizada em 04 de maio de 2026) tem como objetivo apresentar a concepção do sistema de visão computacional proposto pelo grupo Codexis: uma solução baseada em YOLOv8 capaz de identificar trabalhadores e seus EPIs em tempo real, gerando alertas, logs e KPIs voltados à gestão de RH e CIPA.

Public

1.2 Requisitos mínimos

Para esta fase do Challenge, foram definidos os seguintes requisitos mínimos, delimitando o que está dentro e fora do escopo do MVP:

Dentro do escopo

- Detecção de humanos e EPIs (capacete, colete, máscara e óculos de proteção);
- Pipeline de coleta, anotação (labeling) e treinamento de modelo YOLOv8;
- API em Flask para inferência do modelo, com geração de logs em PDF/CSV;
- Front-end web com transmissão de câmera ao vivo, gerenciamento e dashboard de KPIs;
- Demonstração funcional com câmera USB ou vídeo simulado de chão de fábrica.

Fora do escopo (fronteiras da fase)

- Integração física com PLC, esteiras ou catracas industriais;
- Reconhecimento facial ou identificação biométrica de funcionários;
- Hospedagem em nuvem com disponibilidade 24x7 em produção;
- Aplicativo mobile nativo,com foco do MVP é a interface web;
- Conformidade plena com a LGPD em ambiente real de operação.

2. Machine Learning

2.1 Desenvolvimento

2.1.1 Revisão teórica

A frente de Machine Learning parte da experiência já validada pelo grupo no Challenge Aché 2025, reaproveitando uma stack de visão computacional testada anteriormente: detecção de objetos com YOLOv8, back-end em Flask e geração automatizada de logs. Essa continuidade técnica reduz o risco de implementação e permite concentrar o esforço da fase na adaptação do modelo ao novo domínio segurança ocupacional industrial.

2.1.2 Dados

O dataset é composto por imagens de trabalhadores em ambiente simulado de chão de fábrica, anotadas com bounding boxes para as classes de interesse: Person (pessoa), Hardhat / NO-Hardhat (capacete), Mask / NO-Mask (máscara) e Safety Vest / NO-Safety Vest (colete). A anotação é realizada na ferramenta Roboflow, que também é utilizada para versionamento e exportação do dataset no formato compatível com o treinamento YOLOv8. O volume final de imagens e a divisão entre os conjuntos de treino, validação e teste serão detalhados e consolidados nas próximas entregas (E2 e E3), à medida que o dataset for ampliado com imagens reais de chão de fábrica.

Public

A análise geral do modelo nesta fase é qualitativa, baseada nos testes exploratórios de inferência apresentados na demonstração (seção 2.3), servindo como baseline para as rodadas de retreino previstas no cronograma.

2.1.3 Metodologia

O algoritmo escolhido foi o YOLOv8 (Ultralytics), por seu equilíbrio entre velocidade de inferência e precisão, adequado a aplicações de detecção em tempo real critério decisivo para um sistema de monitoramento contínuo de EPIs. O modelo é fine-tuned a partir de pesos pré-treinados, com a arquitetura de detecção de objetos original mantida e a camada de classificação ajustada para as classes específicas do projeto.

A estratégia de treinamento planejada segue as etapas: (1) setup e inferência inicial do modelo base; (2) labeling e definição de bounding boxes no dataset; (3) treino e testes do modelo de detecção de EPI; (4) treino de um modelo complementar de detecção de risco ergonômico; (5) treino de um modelo de detecção de zona de perigo; (6) ciclo de testes e retreino dos modelos; (7) validação; e (8) otimização de latência do sistema para uso em tempo real.

2.1.4 Ferramentas

- Ultralytics YOLOv8 : arquitetura e treinamento do modelo de detecção;
- OpenCV: captura e pré-processamento de frames de vídeo;
- Python :linguagem principal do pipeline de ML;
- Roboflow: anotação, organização e versionamento do dataset.

2.1.5 Pipeline de implementação

O fluxo de inferência segue quatro camadas, conforme o fluxograma do processo (seção 4.3.1): a camada de Sistema de Captura conecta-se à câmera e captura o frame do vídeo; a camada de Machine Learning executa o modelo treinado e retorna a classificação (EPI detectado ou não); a camada de Interface Visual atualiza a bounding box com o rótulo OK ou NOK e, em caso de não conformidade, dispara alerta visual e sonoro; por fim, a camada de Sistema de Log registra os dados, salva o frame localmente e armazena as informações no banco de dados, retroalimentando os gráficos do dashboard.

2.2 Resultados

2.2.1 Métricas de avaliação

O desempenho do modelo é acompanhado por meio de uma matriz de confusão por classe de EPI, disponível no dashboard, que evidencia a taxa de acerto do modelo para cada categoria detectada (capacete, colete e óculos).

2.2.2 Desempenho do(s) modelo(s)

Nos testes exploratórios de inferência (demonstração em tempo real via webcam), o modelo apresentou os seguintes níveis de confiança: detecção de pessoa (0,72–0,85), capacete presente (0,86) e ausente (0,89), máscara presente (0,85) e ausente (0,92), e colete ausente (0,54–0,82). Na matriz de confusão do dashboard, as classes capacete e óculos atingiram 95% de acerto, enquanto a classe colete atingiu 82%.

2.2.3 Análise de erros

A matriz de confusão indica maior confusão entre as classes colete e óculos (13% de erro cruzado), sugerindo sobreposição visual entre esses elementos em determinados ângulos de câmera ou distância. Esse ponto foi identificado como prioridade para as próximas rodadas de retreino, com ampliação do dataset e maior diversidade de poses e distâncias da câmera.

2.3 Evidências

2.3.1 Visualizações de suporte

O dashboard reúne quatro visualizações principais de suporte à análise do modelo: um gráfico de análise composta (série temporal de detecções ao longo do dia), um gráfico de detecções em formato de rosca (prevalência de objetos detectados por classe), um radar de eficiência operacional (precisão da IA, estabilidade, conexão com o hardware, tempo de resposta e segurança/EPI) e a matriz de confusão por classe.



Figura 1 - Dashboard com central de logs, gráficos de detecção e matriz de confusão.

2.3.2 Exemplos práticos

As imagens a seguir mostram exemplos reais de inferência do modelo em vídeo capturado por webcam, com bounding boxes e respectivos scores de confiança para cada classe detectada.

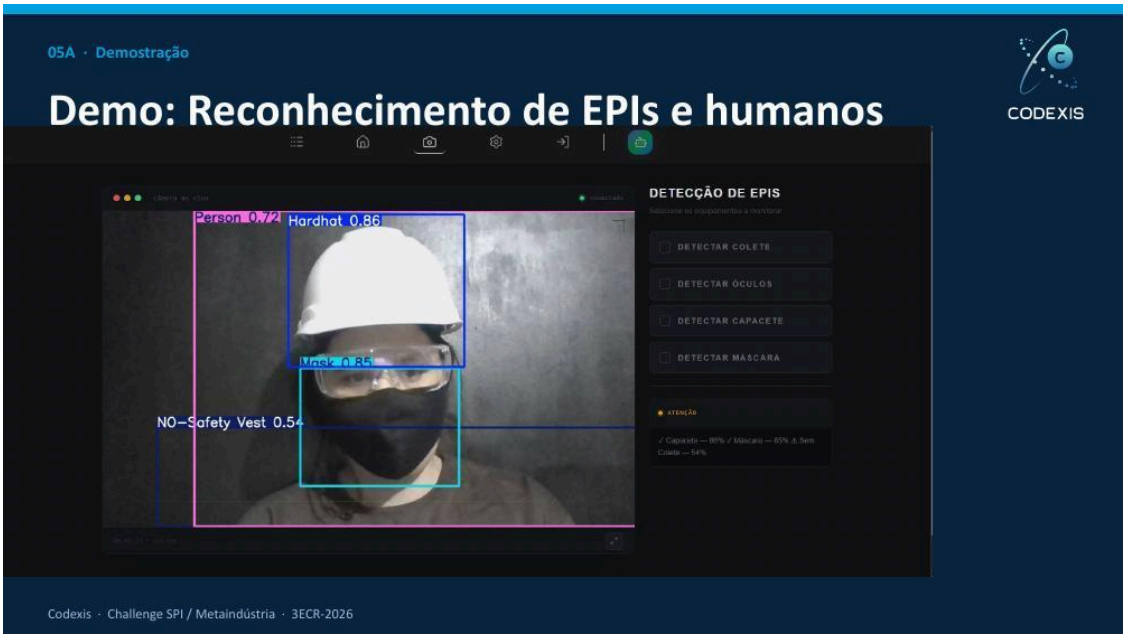


Figura 2 - Detecção de capacete e máscara presentes, colete ausente (NO-Safety Vest).



Figura 3 - Detecção de ausência de capacete, máscara e colete (cenário de não conformidade)

Public

2.3.3 Logs e reprodutibilidade

Cada evento de detecção é registrado com timestamp na central de logs do dashboard (ex.: “Operário detectado sem capacete na Zona B”, “Detecção de óculos de proteção confirmada”), permitindo rastreabilidade e geração de relatório via botão dedicado. O versionamento do código do modelo, do dataset e das configurações experimentais é mantido no GitHub do grupo, e o planejamento das rodadas de treino é documentado no Miro (roadmap do projeto).

3. Frontend

3.1 Desenvolvimento

3.1.1 Revisão teórica

A frente de Frontend reaproveita padrões de interface já validados no Challenge Aché 2025, como telas de transmissão ao vivo e telas de gerenciamento de API, adaptando-os ao novo contexto de segurança ocupacional industrial.

3.1.2 Requisitos e escopo da interface

Para esta fase, o escopo de interface contempla: definição de identidade visual no Figma; desenvolvimento da tela web principal (index); implementação do dashboard inicial com aba de logs; atualização contínua de gráficos e elementos visuais conforme novos dados de inferência; e a tela de câmera dedicada à zona de perigo.

3.1.3 Arquitetura e componentes

A interface é organizada em torno de dois blocos principais: (1) a tela de câmera ao vivo, com sobreposição das bounding boxes e um painel lateral de “Detecção de EPIs”, no qual o usuário seleciona via checkboxes quais equipamentos deseja monitorar (colete, óculos, capacete, máscara), além de um painel de “Atenção” com os percentuais de confiança das detecções; e (2) o dashboard de gestão, estruturado em cards - central de logs, análise composta, gráfico de detecções, eficiência operacional e matriz de confusão - com um botão de geração de relatório.

3.1.4 Integração com backend

O front-end consome a API Flask para receber, em tempo real, as classificações produzidas pelo modelo de ML e alimentar tanto o painel de detecção ao vivo quanto os gráficos e a central de logs do dashboard, tratando os diferentes estados de resposta (detecção confirmada, ausência de EPI, erro de conexão).

Public

3.1.5 Ferramentas

- React: construção dos componentes de interface;
- HTML/CSS: estruturação e estilização das telas;
- Figma: prototipação e definição da identidade visual.

3.1.6 Pipeline de implementação

O desenvolvimento seguiu a sequência: definição de identidade visual no Figma; desenvolvimento da tela web/index; construção do dashboard inicial e da aba de logs; atualização incremental de gráficos e elementos visuais conforme a evolução do modelo; e, por fim, implementação da tela de câmera de zona de perigo.

3.2 Resultados

3.2.1 Critérios de avaliação

A interface foi avaliada de forma preliminar quanto à clareza dos alertas visuais e sonoros de não conformidade, à legibilidade das bounding boxes sobrepostas ao vídeo e à organização das informações no dashboard para consulta rápida por gestores de SST.

3.2.2 Funcionalidades entregues

Nesta fase foram entregues: a tela de câmera ao vivo com detecção e rotulagem em tempo real; o painel de seleção de EPIs a monitorar; o painel de atenção com os percentuais de confiança; e o dashboard completo com os cinco cards de KPIs e a central de logs com botão de geração de relatório.

3.2.3 Análise de problemas

Como pontos de atenção identificados nesta fase, destacam-se a necessidade de testes de usabilidade mais formais com usuários representativos (equipes de RH/CIPA) e o refinamento do comportamento do painel de alertas em cenários com múltiplas não conformidades simultâneas, itens que serão detalhados nas próximas entregas.

3.3 Evidências

3.3.1 Prints e protótipos de telas

As telas finais implementadas refletem o protótipo definido no Figma, com a tela de câmera ao vivo e o painel lateral de detecção de EPIs



.Figura 4 - Tela de câmera ao vivo com painel de detecção de EPIs e painel de atenção

3.3.2 Exemplos práticos

O dashboard de KPIs, apresentado na seção 2.3.1 (Figura 1), ilustra o fluxo completo de uso: consulta de logs, análise de gráficos de detecção e geração de relatório a partir da interface.

3.3.3 Logs e reprodutibilidade

O código do front-end é versionado no GitHub do grupo, com o planejamento de sprints documentado no Miro. A configuração do ambiente de desenvolvimento segue as dependências do React e das bibliotecas de visualização utilizadas nos gráficos do dashboard.

4. Backend

4.1 Desenvolvimento

4.1.1 Revisão teórica

A frente de Backend reaproveita o padrão de arquitetura já validado no Challenge Aché 2025 - API em Flask com persistência em SQLite -, adaptado para receber as inferências do

Public

modelo de visão computacional e expor os dados necessários ao dashboard e à central de logs.

4.1.2 Requisitos e escopo dos serviços

O escopo desta fase para o backend contempla: setup inicial do Flask; criação da API e do banco de dados; implementação de sistema de cadastro/login; integração dos dados de logs com os gráficos do dashboard; e controle das configurações do modelo de ML (parâmetros de inferência e classes monitoradas).

4.1.3 Arquitetura

A arquitetura é composta por uma API Flask que expõe endpoints de inferência e consulta, com persistência em SQLite. Um módulo específico de Sistema de Logs é responsável pelo armazenamento das detecções no banco de dados, pelo armazenamento de frames em arquivo local, pela criação de endpoints dedicados para consulta de logs e pelo registro de eventos relacionados à zona de perigo e a riscos ergonômicos.

4.1.4 Integração com o modelo de ML

O backend recebe as saídas do modelo YOLOv8 (classes detectadas, bounding boxes e scores de confiança) por meio do módulo de inferência, processa essas informações, grava os eventos na base de dados e disponibiliza os dados agregados para os endpoints consumidos pelo dashboard e pela central de logs.

4.1.5 Ferramentas

- Flask - framework da API central;
- SQLite - banco de dados relacional leve para persistência dos logs;
- Python - linguagem base dos serviços de backend.

4.1.6 Pipeline de implementação

O desenvolvimento seguiu duas frentes paralelas: (1) API Central - setup do Flask, criação da API e do banco de dados, sistema de cadastro/login, integração dos dados de logs com os gráficos e controle das configurações do modelo de ML; e (2) Sistema de Logs - implementação do armazenamento no banco de dados, do sistema de frames em arquivo local, da criação de endpoints para logs e do registro de logs específicos para zona de perigo e riscos ergonômicos.

4.2 Resultados

4.2.1 Métricas de avaliação

As métricas de desempenho dos serviços - tempo de resposta dos endpoints, disponibilidade e taxa de erro - serão mensuradas de forma sistemática a partir da fase E2,

quando a API estiver integrada de ponta a ponta com o pipeline de inferência e o dashboard em produção de demonstração.

4.2.2 Desempenho dos serviços

Nesta fase inicial, o foco esteve na criação da estrutura da API e do banco de dados; testes de carga e performance serão realizados e reportados nas próximas entregas, conforme o cronograma do projeto.

4.2.3 Análise de erros

Não há, até o momento, falhas críticas documentadas nos serviços de backend, uma vez que a fase concentrou-se na estruturação inicial da API e do banco de dados. A análise de erros será aprofundada à medida que o volume de requisições aumentar nas próximas fases.

4.3 Evidências

4.3.1 Visualizações de suporte

O fluxograma abaixo representa o fluxo completo do processo, com destaque para a camada de Sistema de Log, responsável por registrar os dados de detecção, salvar os frames e armazenar as informações no banco de dados.

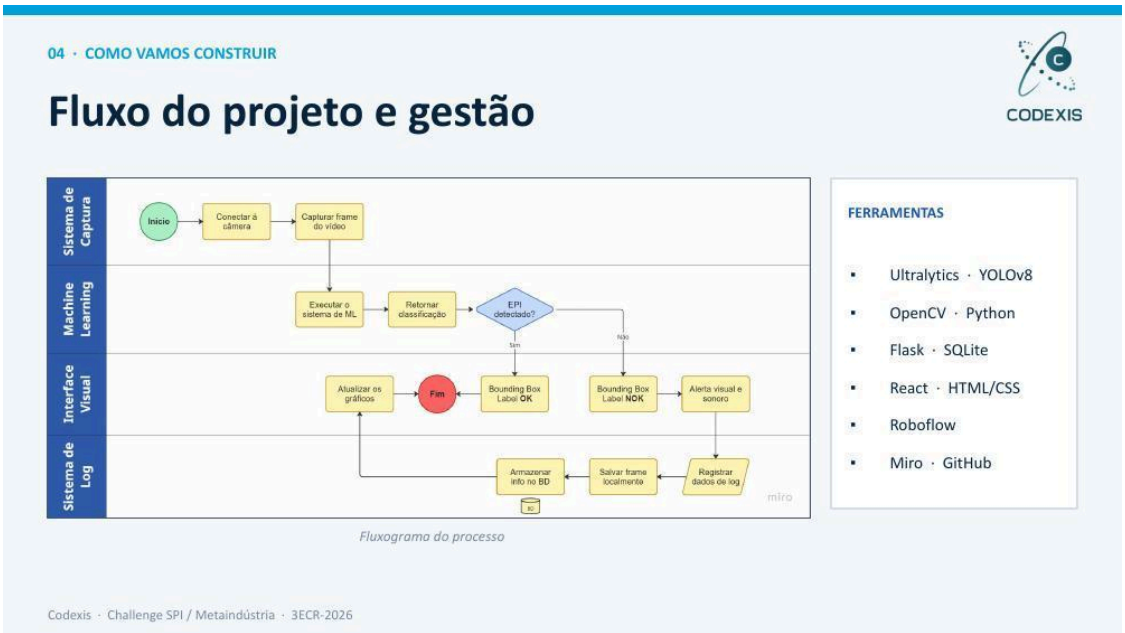


Figura 5 - Fluxograma do processo: captura, inferência, interface visual e sistema de log.

4.3.2 Exemplos práticos

A central de logs exibida no dashboard (Figura 1, seção 2.3.1) é alimentada diretamente pelos endpoints da API, evidenciando o funcionamento integrado entre backend, modelo de

Public

ML e front-end - cada linha de log corresponde a um evento processado e persistido pelo backend.

4.3.3 Logs e reprodutibilidade

O código do backend é versionado no GitHub do grupo. A configuração do ambiente, variáveis e parâmetros do banco de dados serão documentados em detalhe no repositório à medida que o setup evoluir nas próximas entregas.

Public

5. Conclusão

5.1 Próximos passos

O grupo Codexis mapeou as seguintes entregas para dar continuidade ao projeto: fine-tuning do modelo YOLOv8 com a inclusão de novas classes de EPI (óculos, luvas, botas, entre outros); integração completa do sistema de logs com o dashboard e os KPIs; e desenvolvimento de um modelo adicional de detecção de postura, voltado à identificação de risco ergonômico e invasão de zona de perigo.

Essas entregas estão organizadas no roadmap do Challenge SPI/Metaindústria em quatro marcos: E1 (maio, esta entrega) - reconhecimento de EPIs e pessoas, com dashboard inicial; E2 (maio) - API, sistema de logs e modelo de detecção de postura/zona de trabalho; E3 (junho) - fine-tuning do modelo de ML e integração geral dos sistemas (ML, front-end e backend); e E4 (julho) - demonstração final e relatório técnico consolidado.

5.2 Considerações finais

A solução proposta pelo grupo Codexis endereça uma dor real e mensurável do setor industrial brasileiro, substituindo a fiscalização manual e reativa por um monitoramento contínuo e automatizado do uso de EPIs. Como perspectivas de evolução do projeto, destacam-se: a adoção de edge computing, com inferência embarcada diretamente em câmeras IP industriais, reduzindo a dependência de infraestrutura centralizada; a melhoria contínua do modelo por meio da ampliação do dataset com imagens reais de chão de fábrica; e a expansão para análise comportamental, cobrindo riscos ergonômicos, aglomerações e invasões de zonas restritas.

Do ponto de vista de impacto prático, a solução tem potencial para reduzir a ocorrência de acidentes de trabalho relacionados a EPIs, diminuir os custos associados à SST e fornecer subsídios objetivos e rastreáveis para a tomada de decisão de equipes de RH e CIPA, substituindo processos hoje baseados em planilhas e inspeções pontuais por dados gerados em tempo real.